NIMBUS SAAS

SQL Analysis Portfolio

Kaira Srivastava

BBA Graduate — Business Intelligence & Data Analytics

Amity University, Noida

About This Project

Nimbus is a fictional project-management SaaS company. This SQL analysis was performed on a relational PostgreSQL database containing three interconnected tables:

- users — 400 user records including signup date, country, and acquisition channel
- subscriptions — 634 subscription events covering plan history, upgrades, and cancellations
- payments — 2,112 payment records with amounts and success or failure status

All queries were written in PostgreSQL and executed in pgAdmin 4. Concepts covered: JOINs, GROUP BY, aggregate functions (SUM, COUNT, AVG, MIN, MAX), CASE WHEN, DATE_TRUNC, TO_CHAR, LEFT JOIN with IS NULL, HAVING, and subqueries.

Database Schema

The three tables are linked through shared ID columns:

users	subscriptions	payments
————————	—————————————	—————————————
user_id (PK) ———	user_id (FK)	payment_id (PK)
full_name	subscription_id (PK) ———	subscription_id (FK)
email	plan_name	user_id (FK)
country	start_date / end_date	payment_date
acquisition_channel	status / cancel_date	amount
signup_date		payment_status

SQL Queries & Analysis

Query 1: Total Revenue by Acquisition Channel + HAVING Filter

Concept: *JOIN + SUM + GROUP BY + ORDER BY + HAVING*

Business Question

Part A: Which marketing channel generates the most total revenue?

Part B: Which channels specifically cross the 500,000 revenue threshold?

Why This Matters

Part A helps the business rank all channels by revenue. Part B uses HAVING to filter only the top-performing channels — unlike WHERE which filters individual rows, HAVING filters entire group totals after aggregation is done.

Part A — Revenue by Channel (GROUP BY + ORDER BY)

```
SELECT users.acquisition_channel,SUM(payments.amount) AS total_revenue
FROM users
JOIN payments ON users.user_id = payments.user_id
WHERE payments.payment_status = 'Success'
GROUP BY users.acquisition_channel
ORDER BY total_revenue DESC;
```

Query Result

	acquisition_channel character varying (100)	total_revenue numeric
1	Organic Search	856818.80
2	Paid Ads	752490.20
3	Referral	569996.40
4	Direct	460265.00
5	Social Media	430080.60
6	Affiliate	211787.80

Part B — Channels Above 500,000 Only (HAVING)

```
SELECT users.acquisition_channel,SUM(payments.amount) AS total_revenue
FROM users
JOIN payments ON users.user_id = payments.user_id
WHERE payments.payment_status='Success'
GROUP BY users.acquisition_channel
HAVING SUM(payments.amount)>500000
ORDER BY total_revenue DESC;
```

Query Result

	acquisition_channel character varying (100)	total_revenue numeric
1	Organic Search	856818.80
2	Paid Ads	752490.20
3	Referral	569996.40

Key Finding: Organic Search generated the highest revenue overall. Only few channels cross the 500,000 threshold, confirming revenue is concentrated in a few high-performing sources.

Query 2: Most Popular Active Subscription Plan

Concept: *WHERE + COUNT + GROUP BY + ORDER BY*

Business Question

Which subscription plan currently has the most active subscribers?

Why This Matters

Tells the business which plan tier most users prefer, informing pricing and product decisions.

SQL Query

```
SELECT plan_name, COUNT (*) AS active_subscribers
FROM subscriptions
WHERE subscriptions.status = 'Active'
GROUP BY plan_name
ORDER BY active_subscribers DESC;
```

Query Result

	plan_name character varying (50)	active_subscribers bigint
1	Free	164
2	Starter	154
3	Pro	139
4	Business	92

Key Finding: The Free plan has the highest active subscriber count, indicating a large unconverted user base — a key opportunity for upgrade campaigns.

Query 3: New User Signups by Month

Concept: *DATE_TRUNC + TO_CHAR + COUNT + GROUP BY*

Business Question

How many new users signed up on Nimbus each month?

Why This Matters

Tracks whether user growth is accelerating, flat, or declining — a core growth metric for any subscription business.

SQL Query

```
SELECT TO_CHAR(DATE_TRUNC ('month',signup_date),'FMMonth YYYY') AS
signup_month, COUNT (*) AS new_users
FROM users
GROUP BY (DATE_TRUNC ('month',signup_date))
ORDER BY (DATE_TRUNC ('month',signup_date)) ASC;
```

Query Result

	signup_month text	new_users bigint
1	January 2024	19
2	February 2024	22
3	March 2024	26
4	April 2024	26
5	May 2024	23
6	June 2024	22
7	July 2024	26
8	August 2024	26
9	September 20...	26
10	October 2024	22
11	November 2024	17

Key Finding: Signups increased steadily throughout the year, with the highest growth in Quarter 4 — suggesting successful late-year marketing or seasonal demand.

Query 4: Active Subscriptions by Country

Concept: *JOIN + WHERE + COUNT + GROUP BY*

Business Question

Which country has the most currently active Nimbus subscribers?

Why This Matters

Identifies the strongest geographic markets, helping the business prioritise where to expand support, localisation, or sales efforts.

SQL Query

```
SELECT users.country, COUNT (*) AS active_subscribers
FROM users
JOIN subscriptions ON users.user_id=subscriptions.user_id
WHERE subscriptions.status = 'Active'
GROUP BY users.country
ORDER BY active_subscribers DESC;
```

Query Result

	country character varying (150)	active_subscribers bigint
1	India	231
2	United States	107
3	United Kingdom	63
4	Canada	50
5	Germany	31
6	UAE	23
7	Australia	23
8	Singapore	21

Key Finding: India leads in active subscriptions, followed by the United States — these two markets should be prioritised for localised growth campaigns.

Query 5: Overall Churn Rate

Concept: *CASE WHEN + COUNT + ROUND (conditional aggregation)*

Business Question

What percentage of all subscriptions have been cancelled?

Why This Matters

Churn rate is the most important health metric for a SaaS business — it directly impacts long-term revenue and retention strategy.

SQL Query

```
SELECT COUNT (*) AS total_subscriptions,
COUNT (CASE WHEN status = 'Cancelled' THEN 1 END) AS cancelled_subscriptions,
ROUND (COUNT (CASE WHEN status = 'Cancelled' THEN 1 END) *100.0 / COUNT (*),2)
AS churn_rate_percent
FROM subscriptions;
```

Query Result

	total_subscriptions bigint	cancelled_subscriptions bigint	churn_rate_percent numeric
1	634	85	13.00

Key Finding: The overall churn rate is approximately 13% — within acceptable range for a growing SaaS product but worth monitoring closely to prevent acceleration.

Query 6: Top 5 Highest-Paying Users

Concept: *JOIN + SUM + GROUP BY + ORDER BY + LIMIT*

Business Question

Who are the five users who have paid the most in total to Nimbus?

Why This Matters

Identifying VIP customers helps the business prioritise retention — losing one high-value user has outsized revenue impact.

SQL Query

```
SELECT users.full_name,users.country,SUM (payments.amount) AS total_revenue
FROM users
JOIN payments ON users.user_id = payments.user_id
WHERE payments.payment_status = 'Success'
GROUP BY users.full_name,users.country
ORDER BY total_revenue DESC
LIMIT 5
```

Query Result

	full_name character varying (100)	country character varying (150)	total_revenue numeric
1	Diya Smith	India	70981.00
2	Fatima Iyer	United States	64783.80
3	Diya Joshi	India	62384.40
4	Ava Wilson	United States	62181.20
5	Ananya Johnson	Australia	55986.00

Key Finding: The top 5 users contribute disproportionately high revenue — these users should be flagged for dedicated account management and renewal outreach.

Query 7: Average Revenue Per Payment by Plan

Concept: *JOIN + AVG + GROUP BY + ROUND*

Business Question

On average, how much does a single payment amount to for each subscription plan?

Why This Matters

Helps evaluate pricing effectiveness — if the Business plan average is only slightly higher than Pro, it may indicate heavy discounting.

SQL Query

```
SELECT subscriptions.plan_name, ROUND (AVG(payments.amount),2) AS
avg_revenue_per_payment
FROM subscriptions
JOIN payments ON subscriptions.user_id=payments.user_id
WHERE payments.payment_status = 'Success'
GROUP BY subscriptions.plan_name
ORDER BY avg_revenue_per_payment DESC;
```

Query Result

	plan_name character varying (50)	avg_revenue_per_payment numeric
1	Business	2955.70
2	Free	2065.75
3	Pro	1566.33
4	Starter	1067.07

Key Finding: Business plan users show the highest average payment, confirming that higher-tier users generate meaningfully more revenue per transaction.

Query 8: Users Who Have Never Made a Payment

Concept: *LEFT JOIN + IS NULL (finding non-matching rows)*

Business Question

Which users have signed up but never made a single payment?

Why This Matters

These free-tier or lapsed users are a high-value target list for upgrade campaigns and conversion email sequences.

SQL Query

```
SELECT users.full_name,users.country
FROM users
LEFT JOIN payments ON users.user_id=payments.user_id
WHERE payments.payment_id IS NULL;
```

Query Result

	full_name character varying (100)	country character varying (150)
1	Rohan Sharma	India
2	Aditya Mehta	United States
3	Liam Verma	UAE
4	Arjun Bose	India
5	Fatima Rao	India
6	Yusuf Joshi	United Kingdom
7	Layla Kumar	United Kingdom
8	Anika Khan	United States
9	Rohan Kumar	India
10	Sophia Sharma	Australia

Key Finding: A significant number of users have never made a payment — representing untapped revenue potential and prime candidates for targeted upgrade offers.

Query 9: Churn Rate by Acquisition Channel

Concept: *JOIN + CASE WHEN + GROUP BY + churn rate per segment*

Business Question

Which acquisition channel has the highest proportion of cancelled subscriptions?

Why This Matters

Reveals whether certain channels attract low-quality users who don't stay — helping the business stop wasting budget on leaky channels.

SQL Query

```
SELECT users.acquisition_channel, COUNT (*) AS total_subscriptions,
COUNT (CASE WHEN subscriptions.status = 'Cancelled' THEN 1 END) AS
cancelled_subscriptions,
ROUND (COUNT (CASE WHEN subscriptions.status = 'Cancelled' then 1 END)*COUNT (*) /
100.0,2) AS churn_rate_percent
FROM users
JOIN subscriptions ON users.user_id=subscriptions.user_id
GROUP BY users.acquisition_channel
ORDER BY churn_rate_percent DESC;
```

Query Result

	acquisition_channel character varying (100)	total_subscriptions bigint	cancelled_subscriptions bigint	churn_rate_percent numeric
1	Organic Search	194	25	48.50
2	Paid Ads	148	18	26.64
3	Referral	113	16	18.08
4	Direct	75	15	11.25
5	Social Media	71	7	4.97
6	Affiliate	33	4	1.32

Key Finding: Paid Ads show the highest churn rate among all channels, suggesting users acquired through paid campaigns are less committed — a signal to refine targeting.

Query 10: Monthly Revenue Trend

Concept: *DATE_TRUNC + TO_CHAR + SUM + GROUP BY (time series)*

Business Question

How has Nimbus's total monthly revenue changed month by month?

Why This Matters

The monthly revenue trend is the most fundamental business health metric — it shows whether the product is growing, stagnating, or declining over time.

SQL Query

```
SELECT TO_CHAR(DATE_TRUNC('month',payments.payment_date),'FMMonth YYYY') AS
revenue_month,ROUND(SUM(payments.amount),2) AS monthly_revenue
FROM payments
WHERE payments.payment_status = 'Success'
GROUP BY revenue_month
ORDER BY monthly_revenue;
```

Query Result

	revenue_month text	monthly_revenue numeric
1	January 2024	19485.60
2	February 2024	39373.40
3	March 2024	59262.60
4	April 2024	91048.00
5	May 2024	107937.20
6	June 2024	117731.40
7	July 2024	133419.20
8	August 2024	157203.60
9	September 2024	168599.60
10	October 2024	197776.60
11	November 2024	220364.40

Key Finding: Monthly revenue shows a consistent upward trend throughout 2024, with the strongest months in Quarter 4 — consistent with the signup growth observed in Query 3.

Query 11: Payment Range by Subscription Plan (MIN + MAX)

Concept: *JOIN + MIN + MAX + GROUP BY*

Business Question

For each subscription plan, what was the single highest and lowest payment ever made?

Why This Matters

Reveals pricing consistency — if a Business plan's lowest payment equals a Starter plan's highest, it indicates incorrect plan assignments or heavy discounting.

SQL Query

```
SELECT subscriptions.plan_name, MIN (payments.amount) AS lowest_payment, MAX
(payments.amount) AS highest_payment
FROM subscriptions
JOIN payments ON subscriptions.subscription_id=payments.user_id
WHERE payments.payment_status = 'Success'
GROUP BY subscriptions.plan_name
ORDER BY highest_payment DESC,lowest_payment ASC;
```

Query Result

	plan_name character varying (50)	lowest_payment numeric	highest_payment numeric
1	Business	399.20	3999.00
2	Pro	399.20	3999.00
3	Free	399.20	3999.00
4	Starter	399.20	3999.00

Key Finding: Each plan shows a clear and distinct payment range with minimal overlap between tiers, confirming that pricing is being applied consistently across the user base.

Query 12: Users Who Paid Above Average (Subquery)

Concept: *Subquery — a query nested inside another query*

Business Question

Which users have made at least one payment above the overall average payment amount?

Why This Matters

Demonstrates subqueries — the inner query calculates the average first, the outer query uses that result dynamically to filter users without hardcoding any value.

SQL Query

```
SELECT users.full_name,users.country
FROM users
WHERE user_id IN (
SELECT user_id
FROM payments
WHERE payments.amount > (SELECT AVG(payments.amount) FROM payments
 WHERE payments.payment_status = 'Success')
AND payments.payment_status = 'Success'
)
```

Query Result

	full_name character varying (100)	country character varying (150)
1	Rohan Chowdhury	United States
2	Vihaan Kumar	India
3	Ava Verma	Singapore
4	Mason Reddy	United Kingdom
5	Lucas Verma	Singapore
6	Aadhya Brown	United States
7	Dev Chowdhury	Germany
8	Fatima Kumar	India
9	Lucas Rao	India

Key Finding: Users paying above average overlap significantly with the top 5 from Query 6, confirming the concentration of revenue among a small high-value segment.